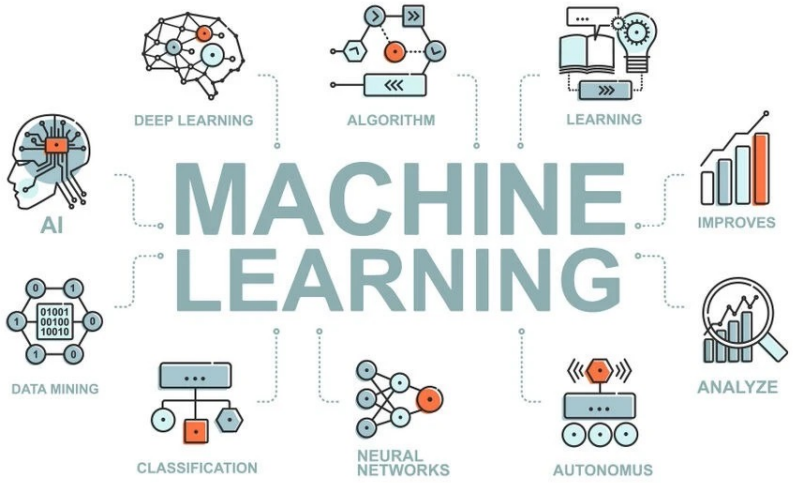
Machine learning fundamentals:

Choosing the right algorithm for biological problems

Jiawei Wang
Lecturer in Computational Biology
Department of Life Sciences
University of Bath
17/06/2026

Outline

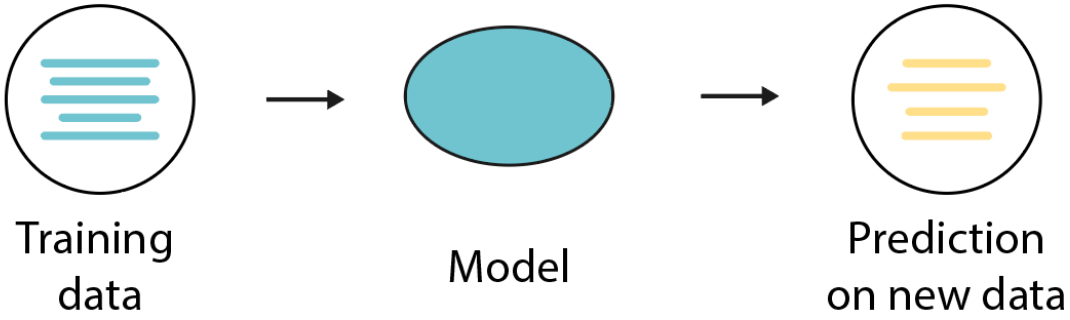
- How do we discover structure in biological data?
- How do we make biological predictions?
- How do foundation models learn biology?
- How do we choose a machine learning method for the right problem?

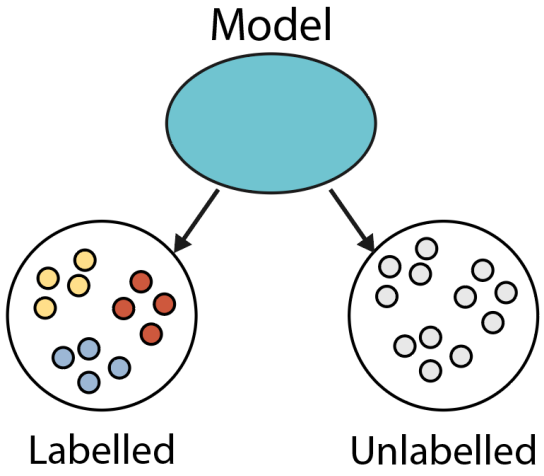


Credit: https://kriviitconsulting.com/machine-learning-mastering-ai-technology/

Recap: What Is Machine Learning?

- Learn patterns from data
- Input: numeric format (vectors, matrices)
- Output:
  - Supervised: prediction with labels
  - Unsupervised: find structure in unlabeled data



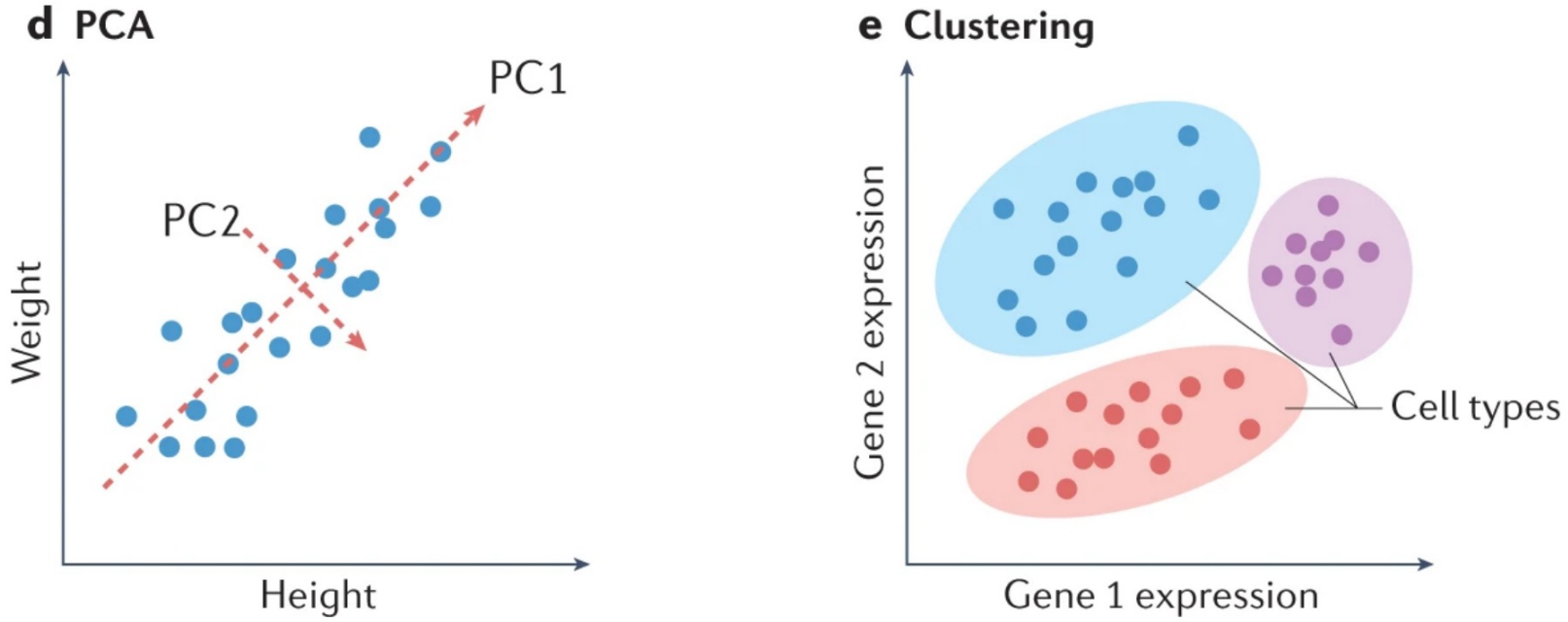


The Core Question

- **"Can we turn a biological problem into a machine learning problem?"**
- Examples:
  - Classification: cancer subtyping, cell type prediction
  - Regression: protein stability, expression prediction
  - Clustering: cell populations
  - Representation learning: protein embeddings from language models (e.g. ESM, ProtT5)
  - More complex: image processing, content generation

Unsupervised Learning

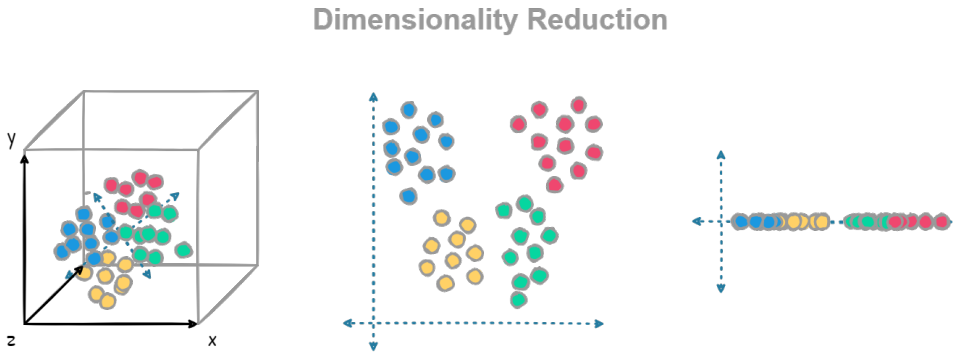
- Finding structure without labels
  - Dimensionality reduction (e.g., PCA, t-SNE and UMAP)
  - Clustering (e.g., k-Means and Hierarchical Clustering)



Credit: https://www.nature.com/articles/s41580-021-00407-0

What Is Dimensionality Reduction?

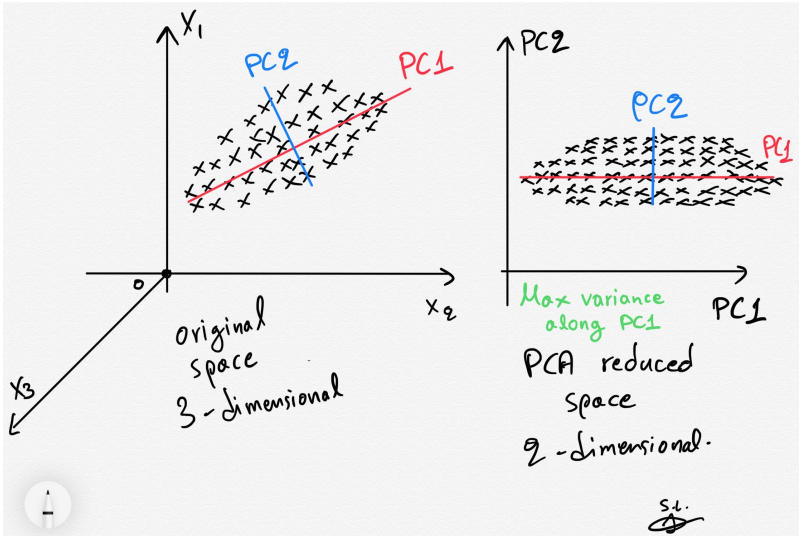
- Goal: project high-dimensional data into a lower-dimensional representation while preserving meaningful structure.
- Used for:
  - Visualization
  - Noise reduction
  - Preprocessing before clustering
  - Feature extraction for ML models

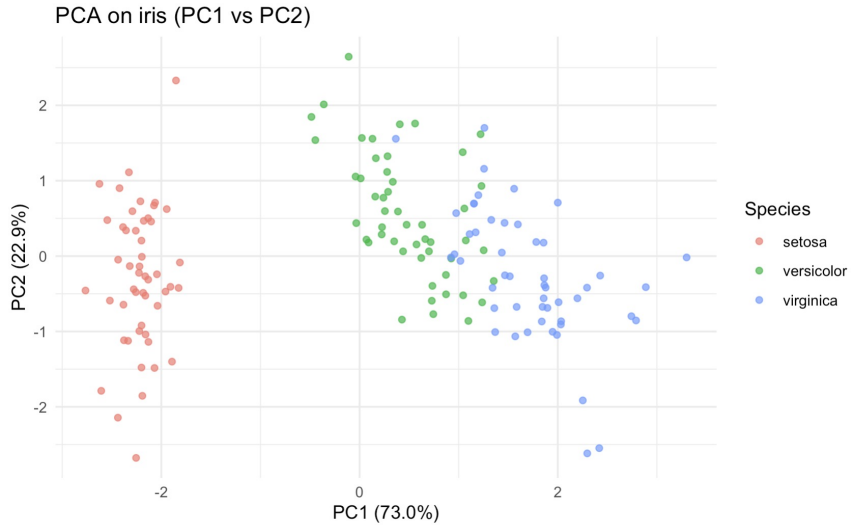


Credit: https://blog.roboflow.com/what-is-dimensionality-reduction/

Principal Component Analysis (PCA)

- Reduces high-dimensional data into a few principal components (PCs).
- Each PC captures the maximum variance in the data.
- PCs are uncorrelated (orthogonal) and ordered by explained variance.
- Helps visualize complex data (e.g. genes, cells, samples) in 2D or 3D.
- PCA is often used as a preprocessing step before t-SNE and UMAP.

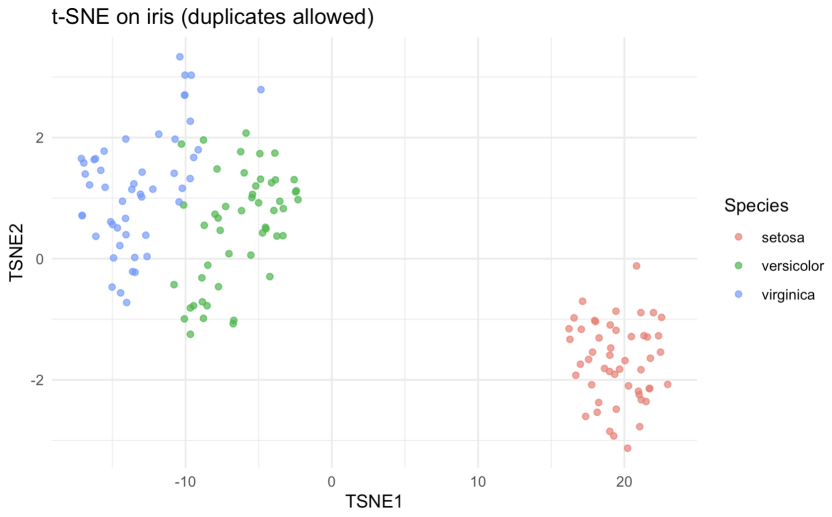




Credit: https://pub.towardsai.net/pca-clearly-explained-when-why-how-to-use-it-and-feature-importance-a-guide-in-python-56b3da72d9d1

t-SNE

- Non-linear dimensionality reduction method.
- Focuses on preserving local structure — points close in **high-dim space** stay close in **2D**.
- Key parameter: **perplexity**
  - Controls balance between local and global structure.
  - Typical range: 5 – 50, depending on dataset size.

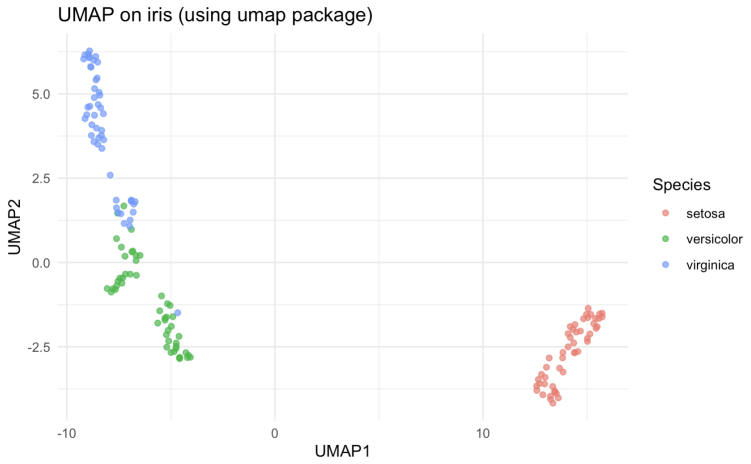



```r
library(Rtsne)

set.seed(123)
tsne_fit <- Rtsne(
  X,                    # numeric features
  dims = 2,             # output dimensions
  perplexity = 30,      # neighborhood size
  theta = 0.5,          # speed-accuracy trade-off
  pca = FALSE,          # skip initial PCA
  check_duplicates = FALSE
)
```

UMAP (Uniform Manifold Approximation and Projection)

- Fast, scalable non-linear dimensionality reduction method.
- Better preserves global structure than t-SNE while maintaining local neighbourhood relationships.
- Key parameters:
  - n_neighbors — size of local neighborhood (smaller = finer detail).
  - min_dist — minimum spacing between points (controls cluster tightness).
  - metric — distance function in original space (e.g. "euclidean").



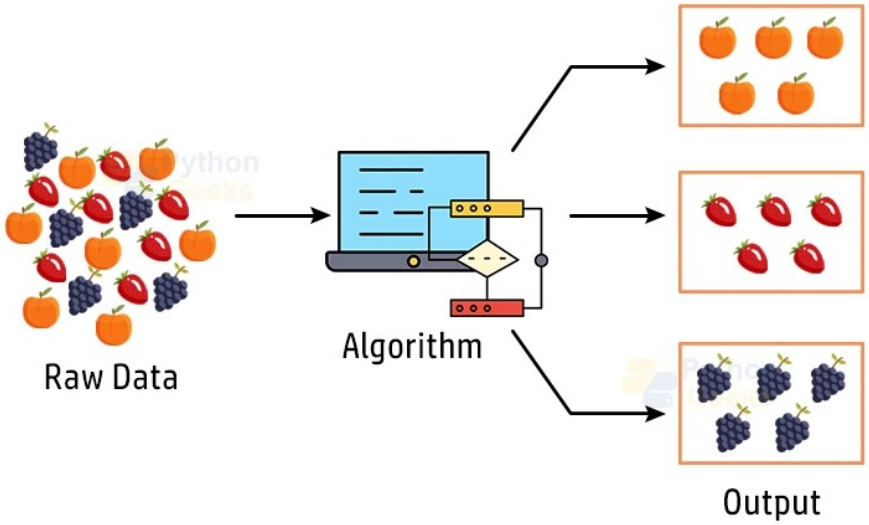

```
library(umap)

set.seed(123)
cfg <- umap.defaults
cfg$n_neighbors <- 15    # local neighborhood size
cfg$min_dist <- 0.1      # controls clustering tightness
cfg$metric <- "euclidean"

umap_model <- umap(X, config = cfg)
```

Clustering Algorithms

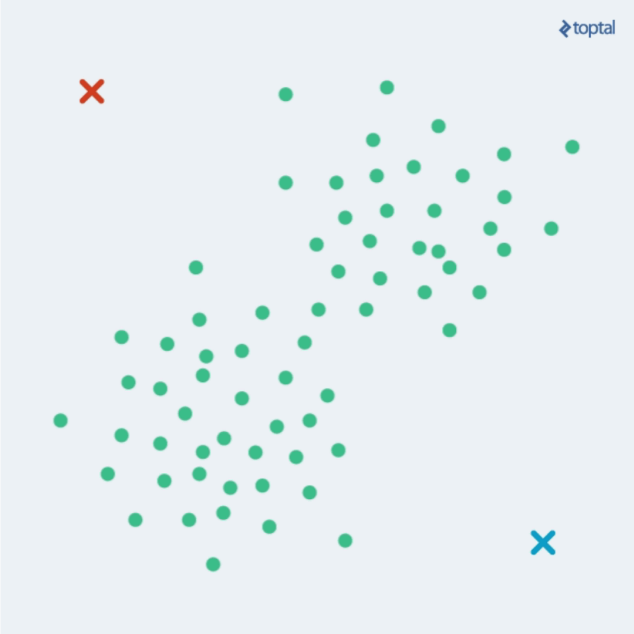
- Two main types:
  - Partitioning methods (k-means)
  - Hierarchical methods (HCA)
- Key difficulty:
  - No “ground truth” labels → evaluation is harder.
  - Internal metrics (e.g., silhouette score) are often used.



Credit: https://pythongeeks.org/clustering-in-machine-learning/

k-Means Clustering

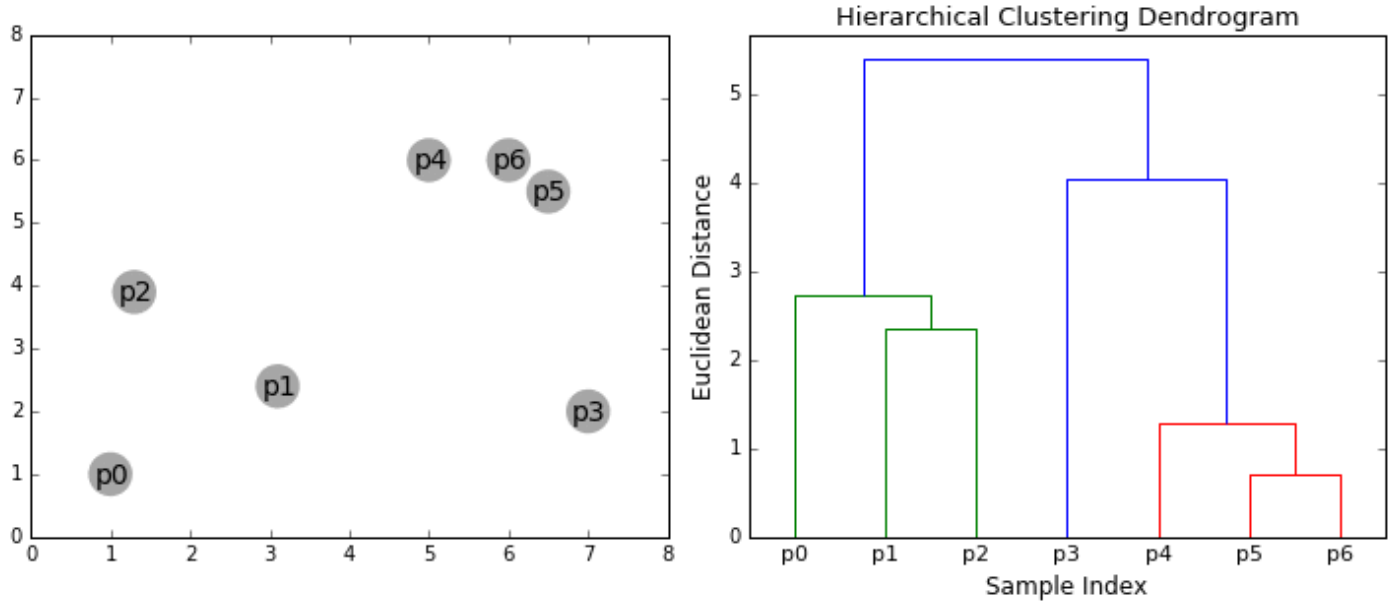
- Groups similar samples into *k* clusters
- Works best for compact, **roughly spherical** clusters
- Minimises distance to the nearest centroid
- Simple and fast, but sensitive to initialisation



Credit: https://pub.towardsai.net/k-means-clustering-from-scratch-4e357ac4716f

Hierarchical Clustering

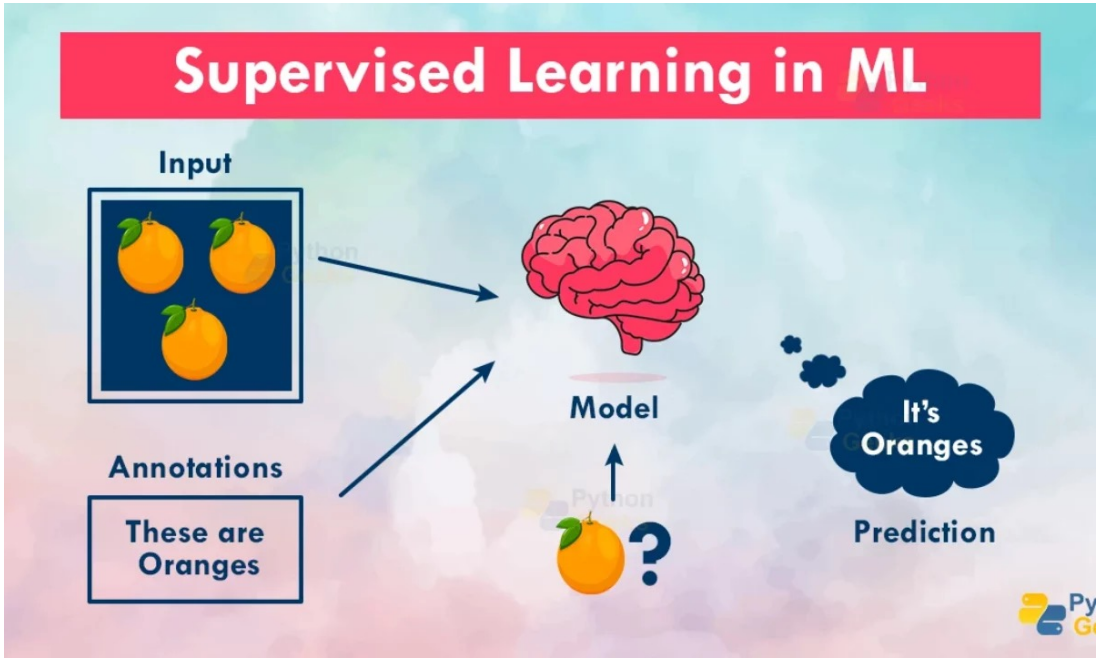
- Builds a tree (dendrogram)
- Doesn’t require predefined number of clusters
- Common in gene expression heatmaps



Credit: https://dashee87.github.io/data%20science/general/Clustering-with-Scikit-with-GIFs/

Supervised Learning

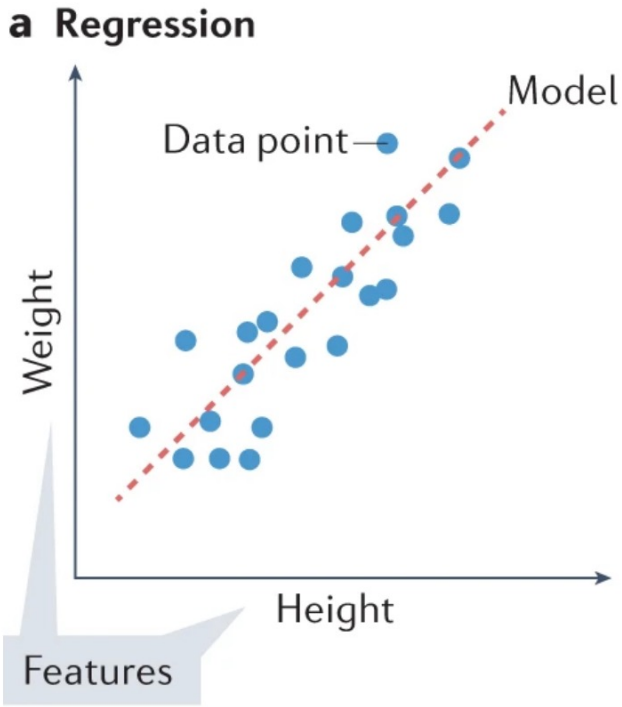
- Goal: learning from labelled data.
  - Predicting classes
  - Predicting continuous outcomes
  - Ranking & scoring tasks



Credit: https://pythongeeks.org/supervised-learning-in-machine-learning/

Linear Regression

- Predicts a continuous outcome from one or more features
- Learns a linear relationship between features and outcome
- Simple, interpretable and widely used baseline model

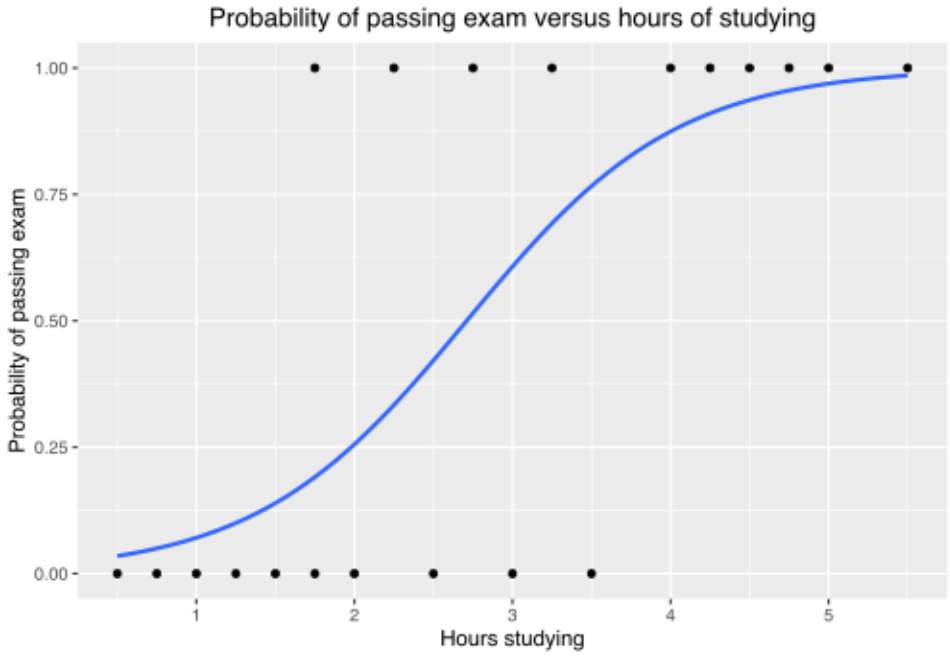



```
Features
   ↓
Linear model
(y = w₁x₁ + w₂x₂ + b)
   ↓
Continuous value
(e.g. weight = 72 kg)
```

Credit: https://www.nature.com/articles/s41580-021-00407-0

Logistic Regression

- Predicts the probability of a binary outcome
- Learns a linear decision boundary in feature space
- Highly interpretable
- Common baseline for classification tasks

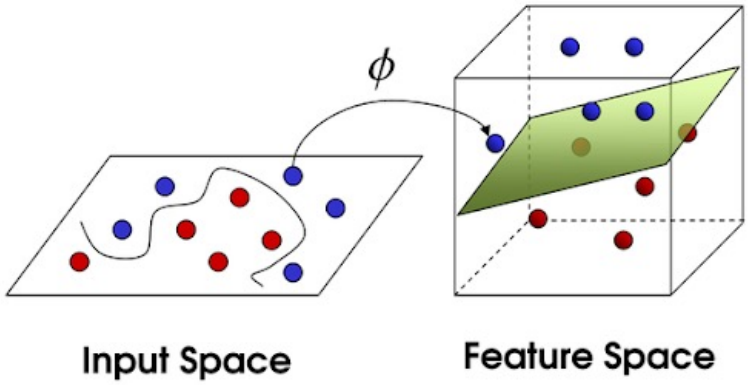



```
Features
   ↓
Linear model
(z = w₁x₁ + w₂x₂ + b)
   ↓
Sigmoid
(converts any value to 0–1)
   ↓
Probability
P(y=1) = 0.92
```

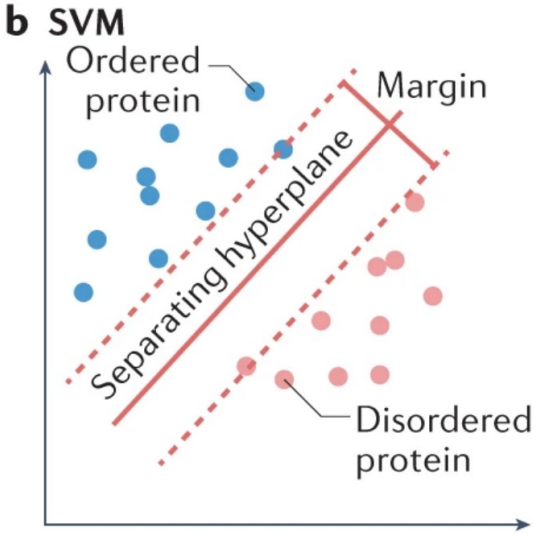
Credit: https://en.wikipedia.org/wiki/Logistic_regression

Support Vector Machine

- Finds the optimal separating hyperplane between classes
- Maximises the margin (distance to nearest points, called support vectors)
- Effective in high-dimensional spaces
- Can model non-linear boundaries using kernels:
  - Linear
  - Polynomial
  - RBF (Gaussian)



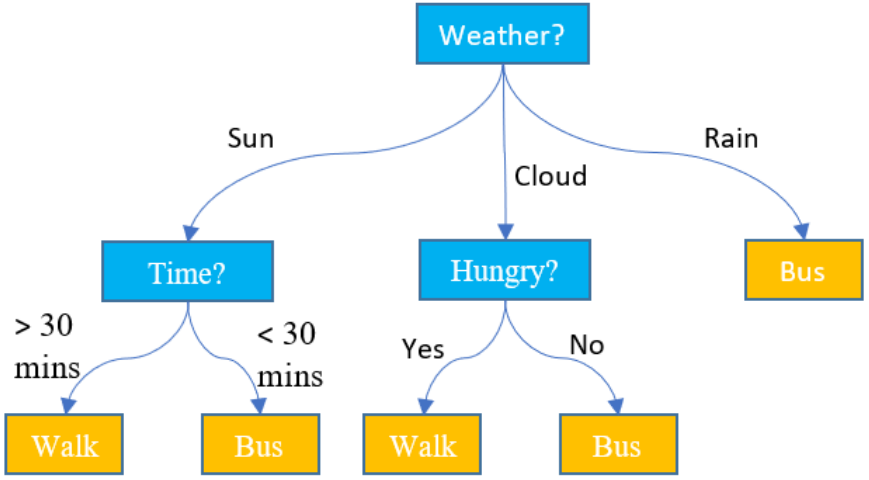
Credit: https://lapix.ufsc.br/ensino/reconhecimento-de-padroes/reconhecimento-de-padroessupport-vector-machines/



Credit: https://www.nature.com/articles/s41580-021-00407-0

Decision Trees

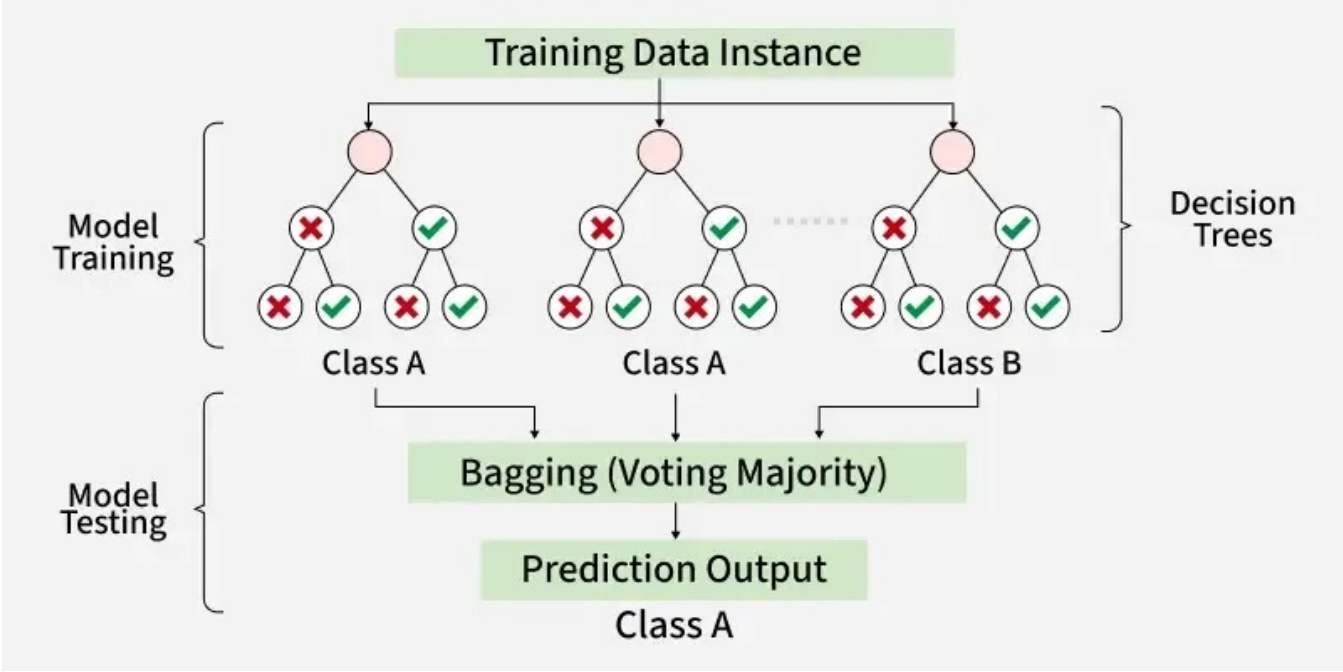
- Intuitive, human-readable step-by-step rules
- Splits data by simple rules
- But unstable & prone to overfitting



Credit: https://www.displayr.com/what-is-a-decision-tree/

Random Forest

- Ensemble of many trees (data sampling + feature sampling)
- Robust & high-performing
- Handles non-linearity
- Provides feature importance

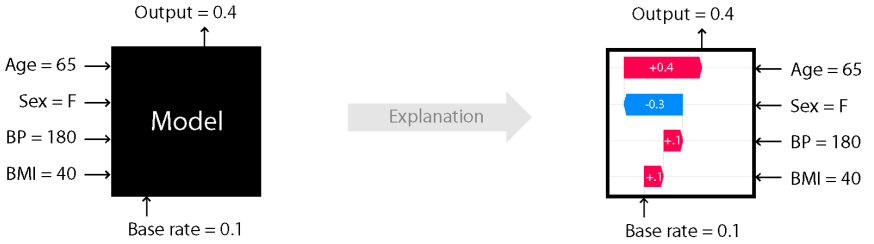


Credit: https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

Feature Importance - Which feature matters most?

- Why do we care?
  - Which genes drive a prediction?
  - Which biomarkers are most informative?
  - Which features can be removed?
- Common approaches
  - Tree-based importance
    - Used in Random Forest
    - Fast and intuitive
  - Permutation importance
    - Shuffle one feature and measure performance drop
    - Often more reliable but slower
  - SHAP values
    - Show which features increase or decrease a prediction
    - Explain why a sample received a particular prediction
    - Widely used in biomedical AI

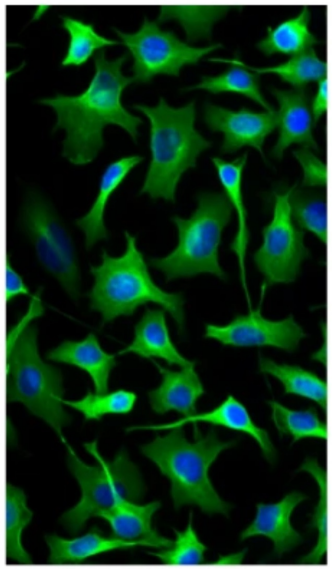
SHAP



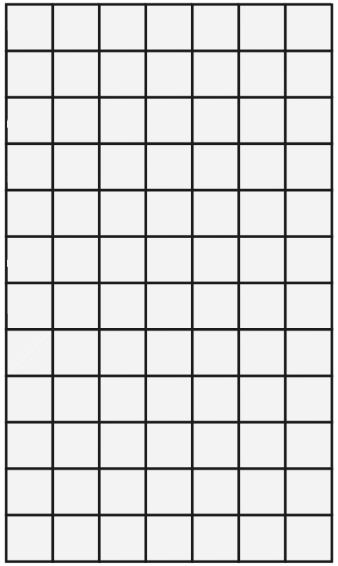
Credit: https://shap.readthedocs.io/en/latest/

U-Net

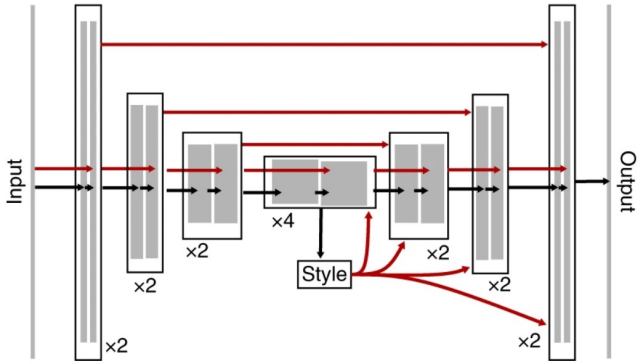
Input image



Pixel matrix



U-Net

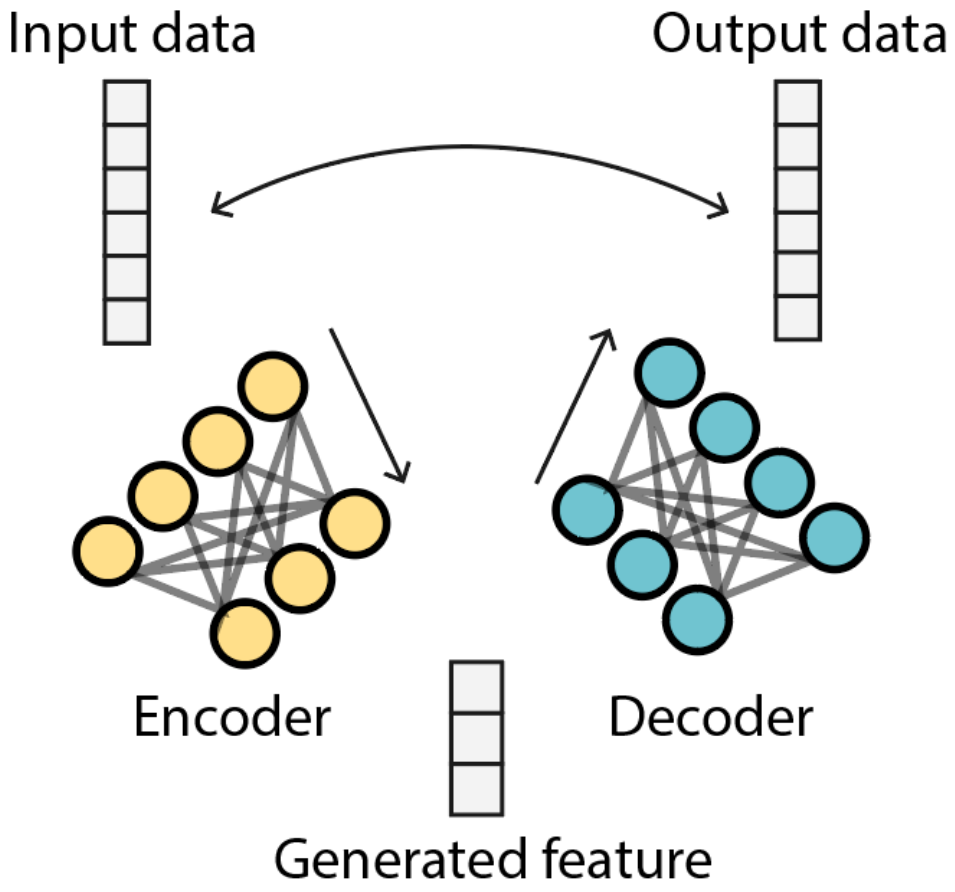


Segmentation



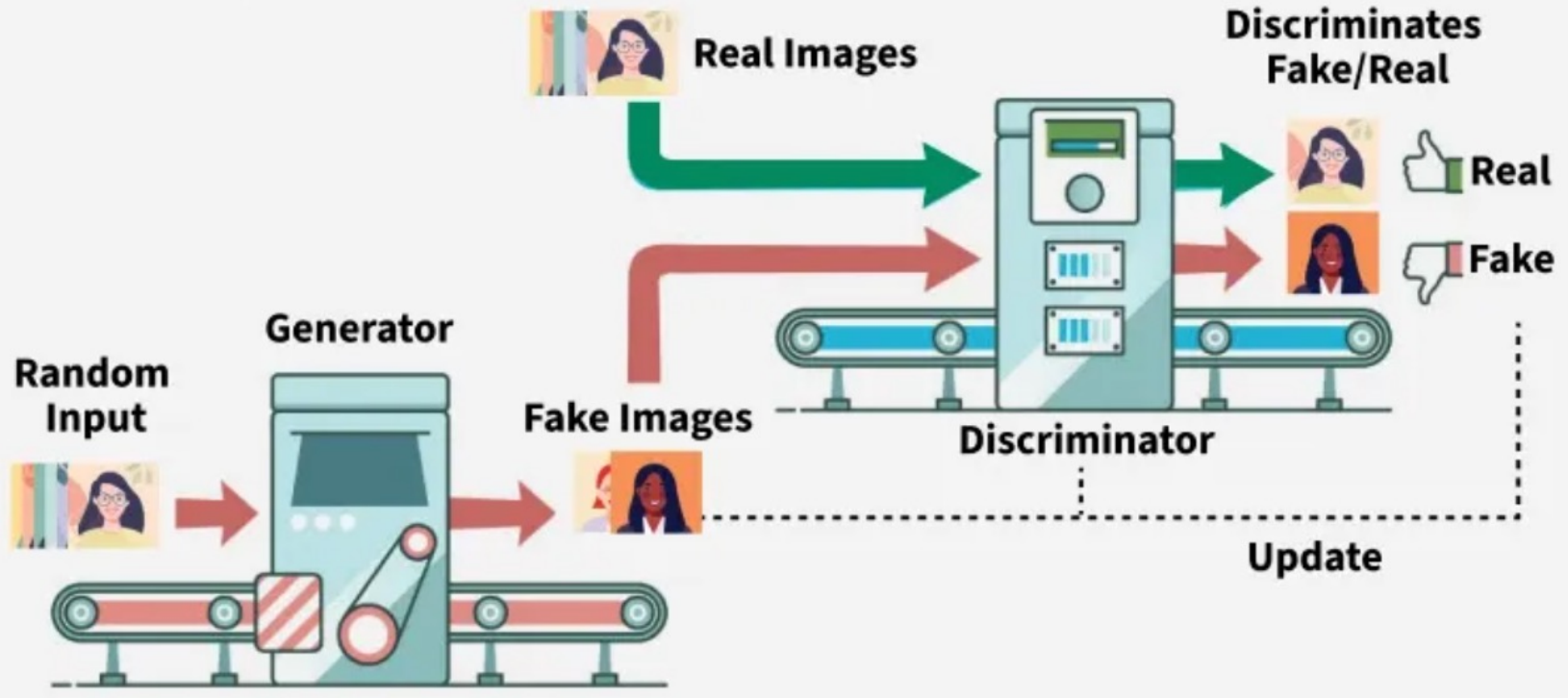
Stringer et al., Nature Methods (2021) - U-Net architecture image is adapted from the paper

Auto-encoder



Can be used in downstream analysis (e.g., dimensionality reduction)

Generative Adversarial Network (GAN)

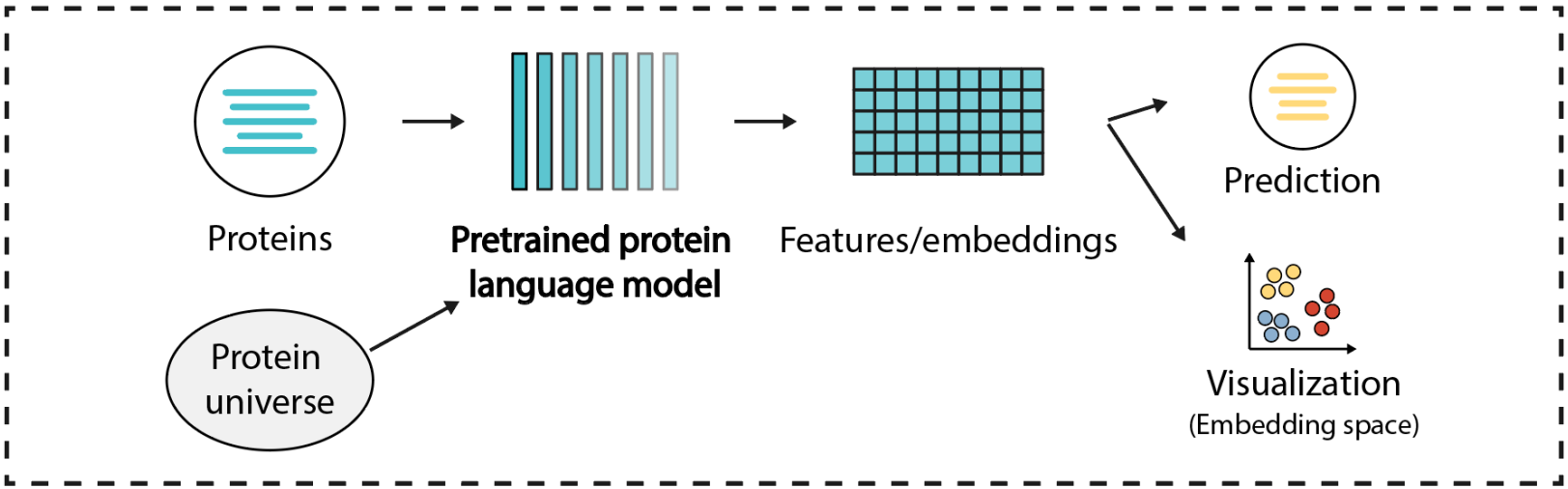


Credit: geeksforgeeks.org/deep-learning/generative-adversarial-network-gan/

Go beyond classification to generate content

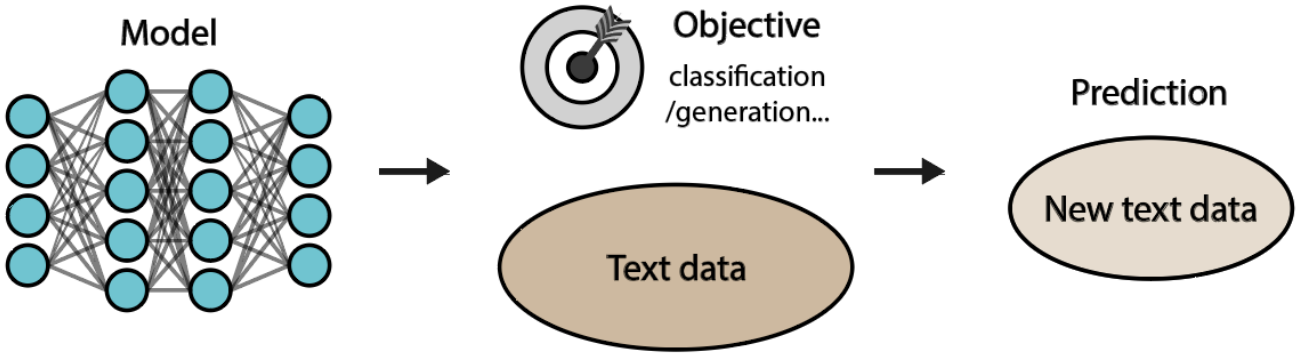
Protein Language Model

- Inspired by NLP (Natural Language Processing) models like BERT, GPT
- Learns protein sequence patterns by **self-supervised training**
- Generates embeddings: numeric representations capturing biological meaning
- Embeddings can be used for classification, clustering, etc.



Why large language models?

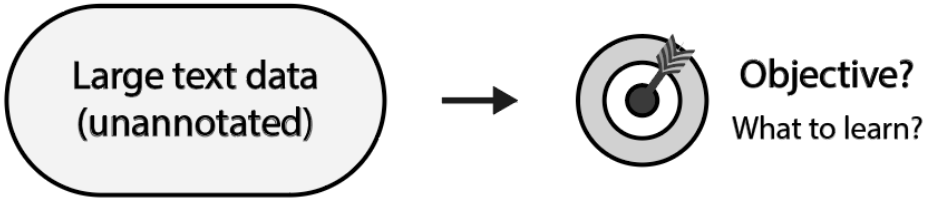
- Traditional approach: Training a **model** to achieve an **objective** using a **dataset**, then making **predictions**.



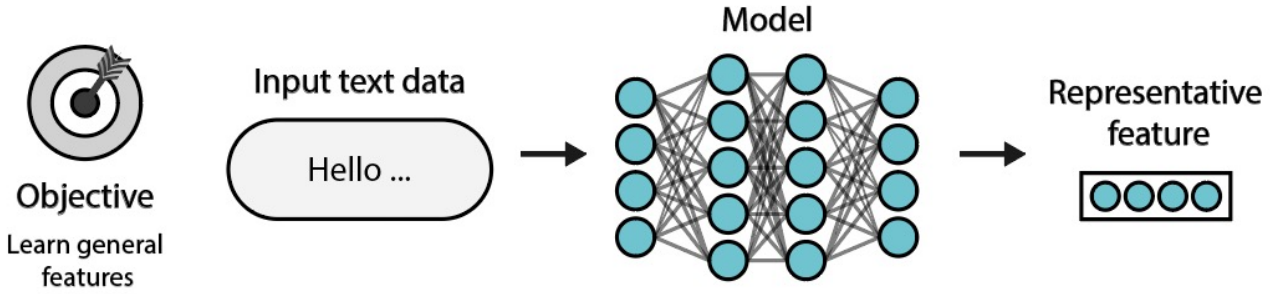
- Limited **annotated** data (This is difficult to get)
- Unlimited **unannotated** data? (This is easy to get)

Large language models

- Take advantage of **unannotated** data
  - Use vast amounts of raw text to extract patterns without manual labelling

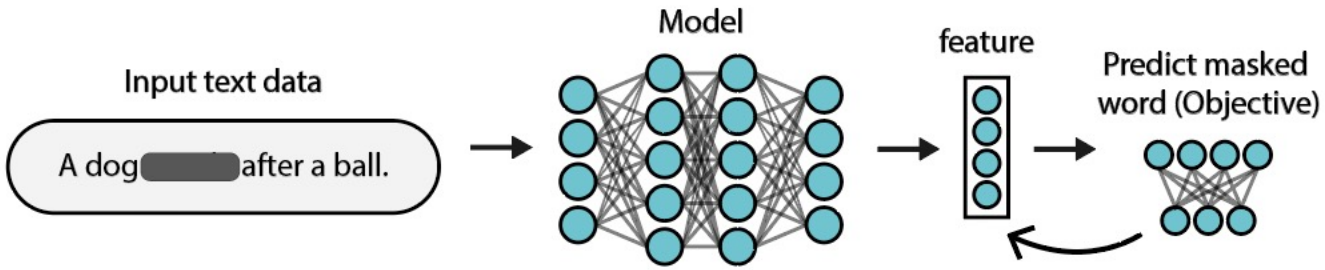


- Purpose:
  - Not to solve a specific task
  - But to build a general understanding of language

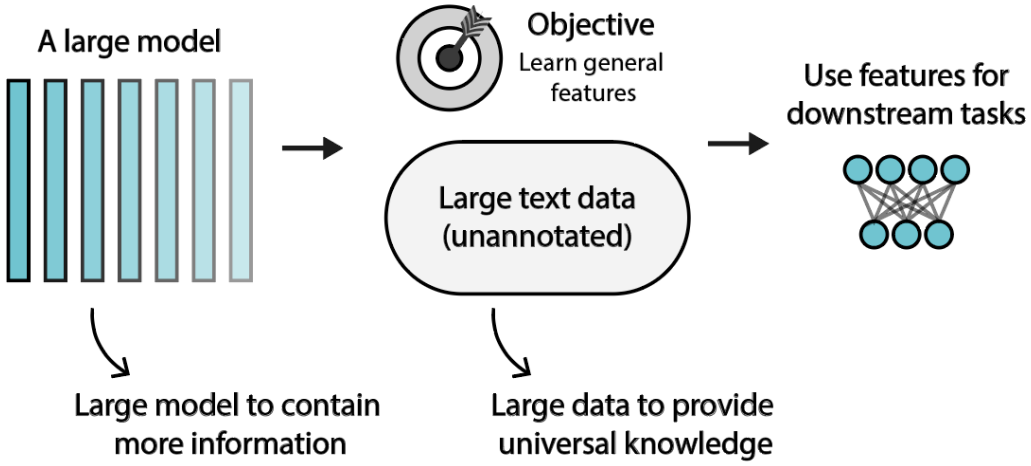


Large language models

- Example objective: Masked Language Modelling

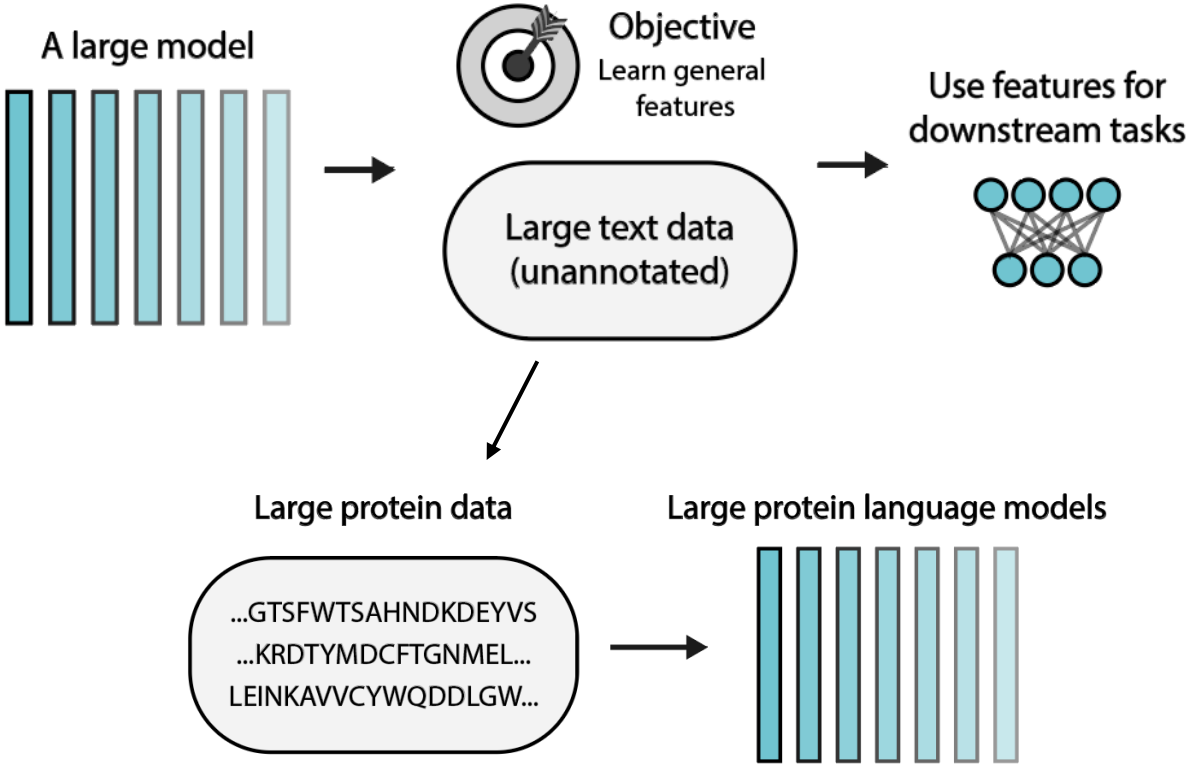


- Framework



From language models to **protein** language models

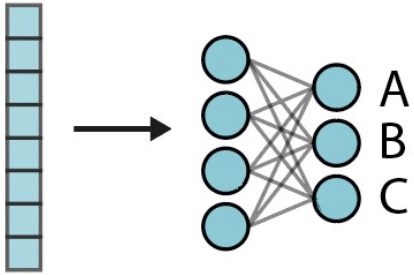
- Idea: protein sequences are similar to sentences



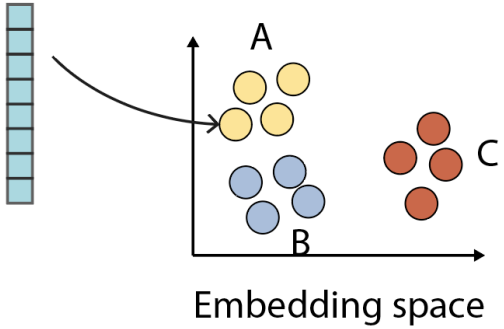
How do we use protein language models?



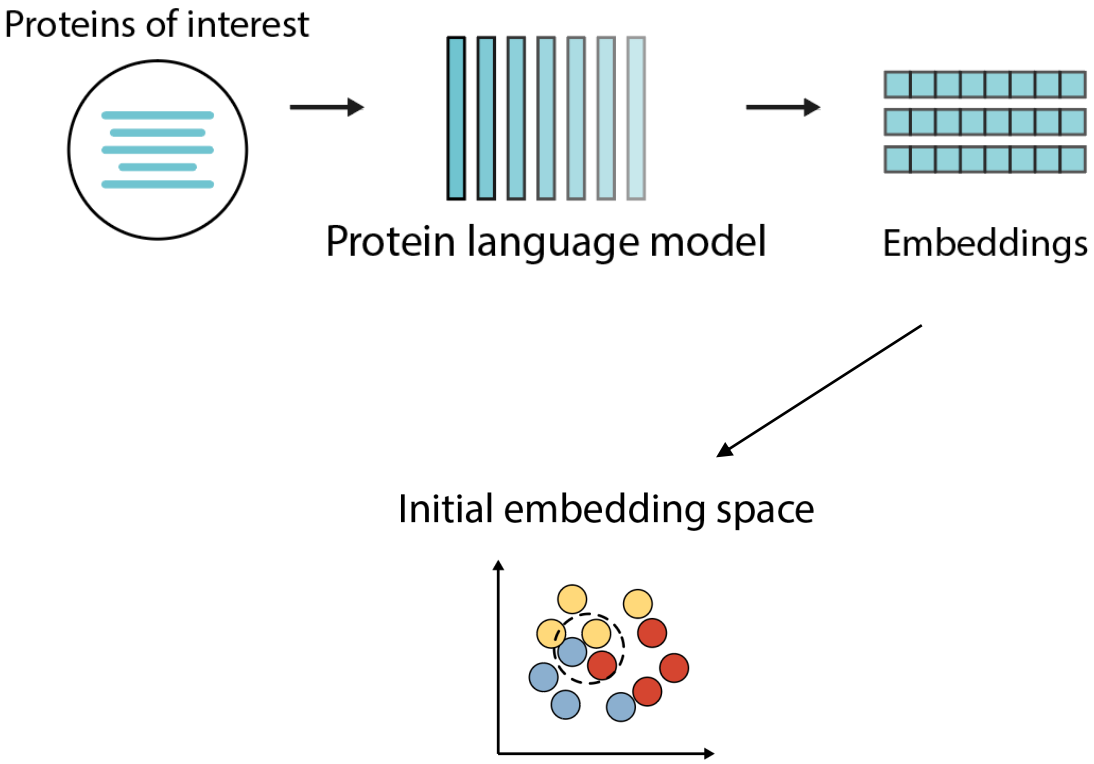
- 1. Classification



- 2. Relationship analysis

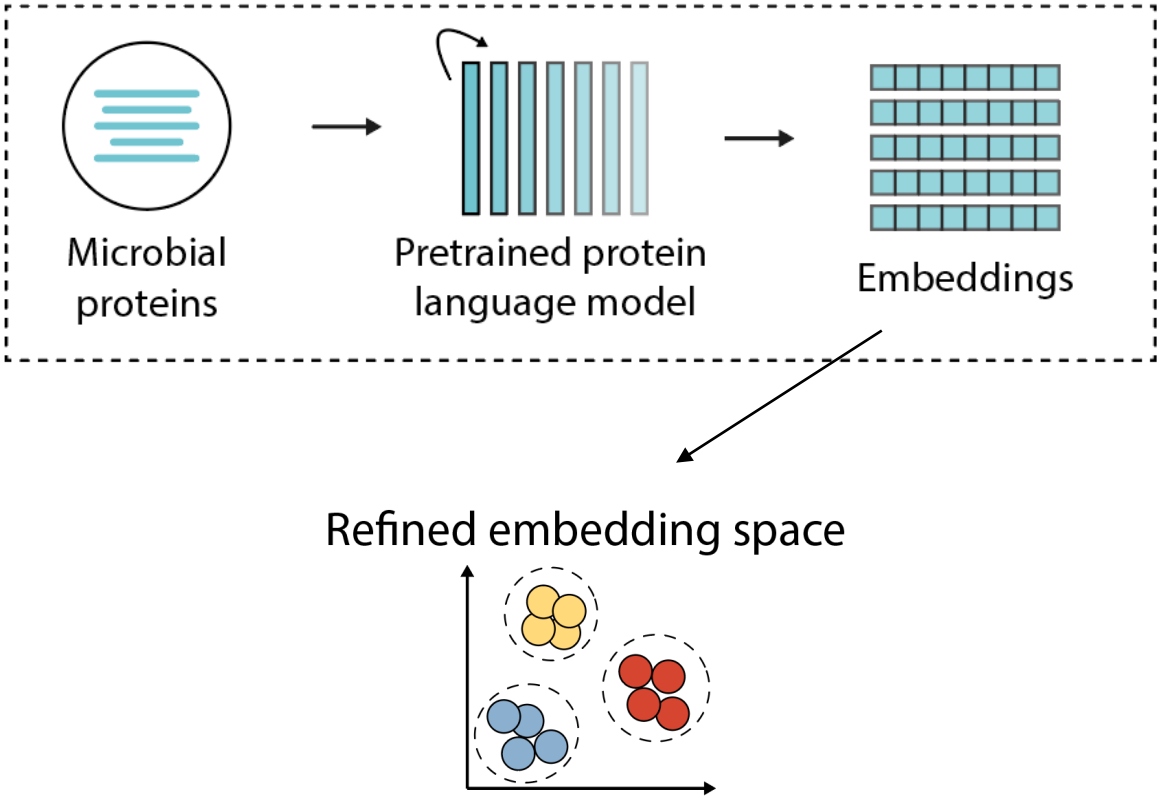


Different ways how we use pretrained language models (1)

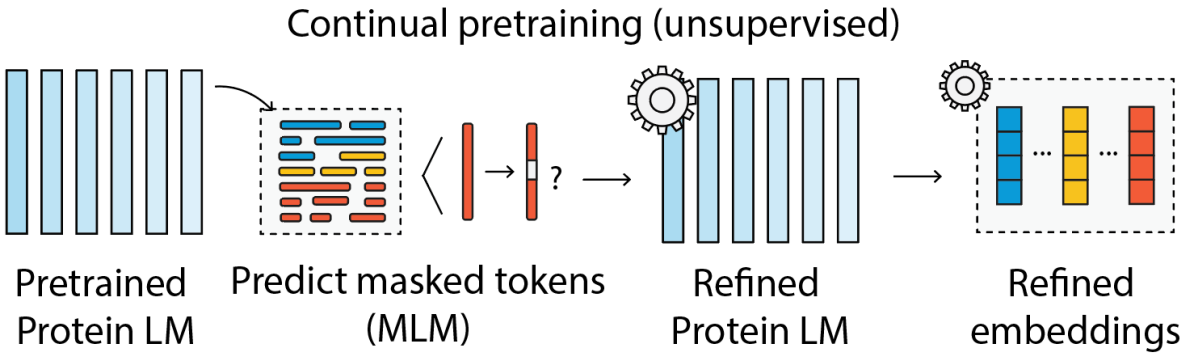


- Get a quick overview of the embeddings' quality to explore relationships among sequences.

Different ways how we use pretrained language models (2)



Continual pretraining



- Enhance sequence understanding to reveal natural information.

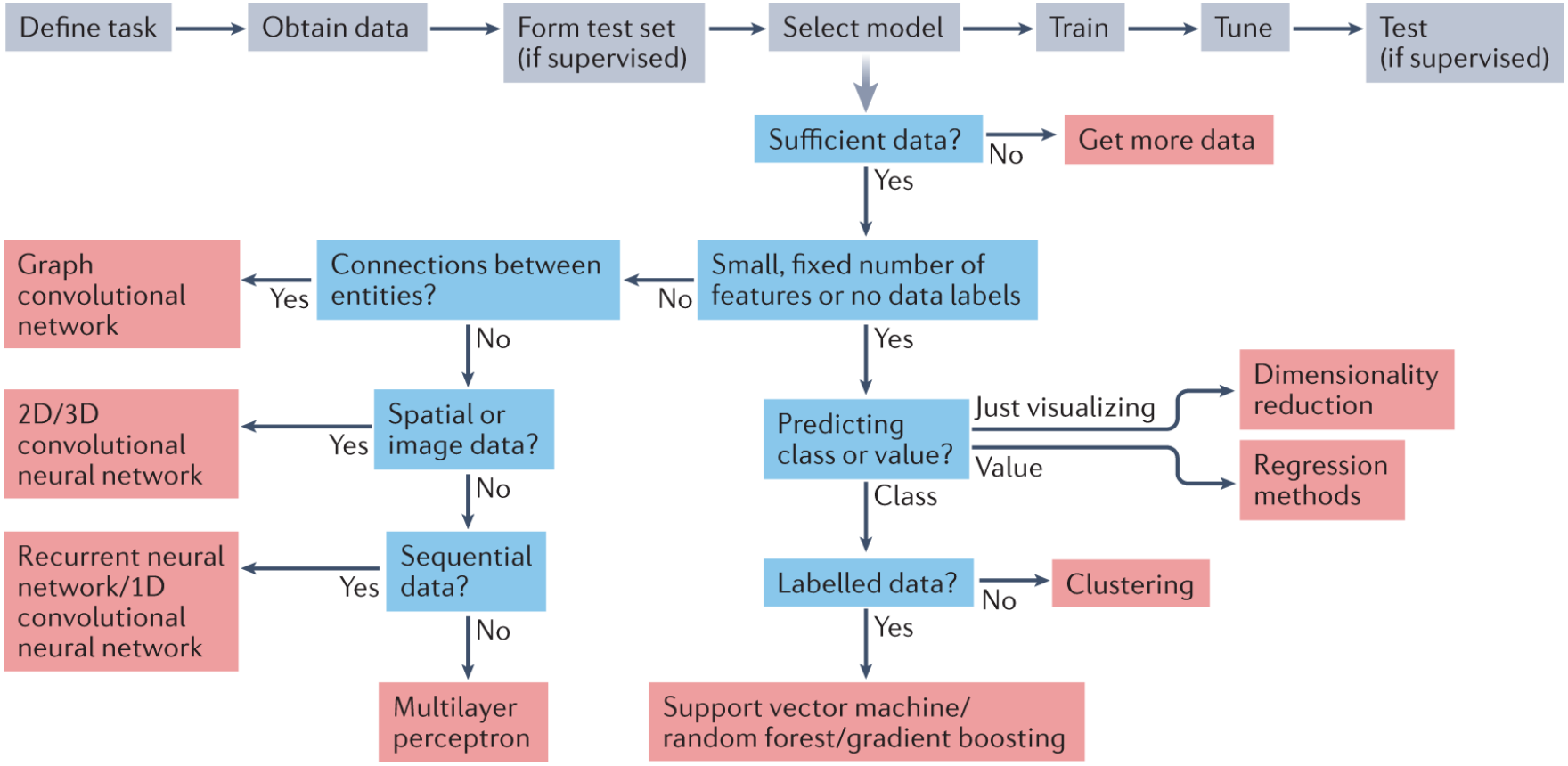
From teaching models biology to learning biology through models

- **Label refinement**: use biology to guide the model
- **Continual pretraining**: let the model explore the unknown

- **Together**: Models learn from biology — and teach us new biology back.

How can we balance what we already know with what we still don't?

How do we choose a machine learning method?



Credit: A guide to machine learning for biologists (*Nature Reviews* at https://www.nature.com/articles/s41580-021-00407-0)

Discussions

- “Think of a biological question & dataset you've seen. Could it be used for ML?”
- What’s the outcome?
- What’s the input?
- What patterns might exist?

Click the following link or scan the code to input your idea:
https://padlet.com/jiaweinohupcc/what-real-world-datasets-have-you-encountered-that-could-be--8tcddl6ilrm5d6zc



Reference

- The content and figures have been credited with links on the relevant slides. These sources are also excellent material for further reading.